

AI Foundations, LLMs, Harnesses, Hermes, OpenClaw, and AI Agents

Prepared for Destynee

Purpose: Give you a clear, beginner-friendly but detailed map of what AI is, what I am, what a harness is, how tools like Hermes/OpenClaw/Claude Code/Codex fit together, and what is happening in the AI-agent market.

Plain-English Executive Summary

AI right now is shifting from **chatbots that answer questions** to **agents that can do work**.

The simplest way to understand the whole thing:

- **LLM = the brain.** It reads language and predicts useful next words, code, plans, or actions.
- **Harness = the body/control system.** It gives the LLM tools, rules, memory, safety limits, and a workspace.
- **Agent = the brain inside the body, trying to complete a goal.** It thinks, uses tools, checks results, and repeats.
- **Coding harness = a special agent body for software work.** It reads code, edits files, runs tests, uses Git, and sometimes opens PRs.
- **Hermes = the harness I am currently running inside.** It gives me Telegram, files, tools, memory, skills, scheduled tasks, subagents, and more.
- **OpenClaw = an agent/gateway/harness ecosystem.** Your older notes referred to me as running on OpenClaw, but this current chat is on **Hermes Agent**.

Analogy:

Imagine you hire a very fast intern.

- The **LLM** is the intern's brain.
- The **harness** is the office: desk, computer, filing cabinet, email, calendar, company rules, manager approvals, and security badge.
- The **agent** is the intern actually working through a task: "make a packet," "fix this bug," "research this topic."
- A **coding harness** is an office built specifically for software: code editor, terminal, tests, GitHub, pull requests.

The model alone is not enough. The surrounding harness decides whether AI can merely talk, or actually act.

Part 1 — Foundational AI Knowledge

1. What is AI?

Artificial Intelligence (AI) means computer systems that perform tasks we normally associate with human intelligence: language, planning, pattern recognition, image understanding, coding, decision support, and problem solving.

AI is not magic. It is math, data, software, and infrastructure.

A simple stack:

1. **Data** — examples the system learned from.
2. **Model** — the trained pattern-recognition machine.
3. **Prompt/context** — what you ask it right now, plus instructions and memory.
4. **Tools** — web search, files, browser, terminal, calendar, email, etc.
5. **Harness/platform** — the system that connects the model to tools safely.
6. **Human oversight** — the person checking whether the output is correct.

2. What is machine learning?

Traditional programming says:

“Here are exact rules. Follow them.”

Machine learning says:

“Here are many examples. Learn the pattern.”

Example:

- Traditional program: “If email contains the words free money, mark spam.”
- Machine learning: “Look at millions of spam and non-spam emails and learn what spam usually looks like.”

3. What is generative AI?

Generative AI creates new content:

- text,
- images,
- audio,

- video,
- code,
- plans,
- summaries.

It does not “copy/paste knowledge” the way a database does. It generates likely outputs from learned patterns.

4. What is an LLM?

An **LLM**, or **Large Language Model**, is an AI model trained on huge amounts of text and code so it can process and generate language.

Examples:

- GPT models from OpenAI,
- Claude models from Anthropic,
- Gemini models from Google,
- Llama models from Meta/open-source ecosystem,
- Qwen, Mistral, DeepSeek, and many others.

Feynman explanation

An LLM is like a person who read a giant library and learned patterns in how people write, explain, argue, code, summarize, and solve problems.

When you ask a question, it does not open a mental folder labeled “truth.” It predicts what response would be useful based on the prompt, its training, and any tools/context it has.

Important limitation

LLMs can be **confidently wrong**. That is called a **hallucination**.

This happens because the model is optimized to generate plausible language, not automatically guaranteed truth. Tools, citations, tests, and verification reduce this risk.

Part 2 — LLM vs Harness vs Agent

5. LLM vs harness: the core difference

LLM

The LLM is the reasoning/text-generation engine.

It can:

- answer questions,
- write explanations,
- draft emails,
- summarize,
- produce code,
- reason through tasks,
- decide which tool should be used next if the harness allows it.

But by itself, an LLM usually cannot:

- read your computer files,
- send Telegram messages,
- schedule reminders,
- run terminal commands,
- browse live websites,
- remember your preferences across sessions,
- edit a code repository,
- verify work with tests.

Harness

A **harness** is the software wrapper around the model.

It provides:

- **Tools:** file read/write, terminal, browser, web, calendars, email, GitHub, etc.
- **Memory:** stored facts or session history.
- **Rules:** system prompts, safety boundaries, user preferences.
- **Execution loop:** model thinks → chooses tool → sees result → continues.
- **Permissions:** approvals for dangerous commands.
- **Interfaces:** Telegram, terminal, web app, IDE, API.
- **Logging:** record what happened.
- **Orchestration:** subagents, scheduled jobs, workflows.

Agent

An **AI agent** is what you get when you combine:

LLM + goal + harness + tools + loop + feedback.

A chatbot may answer: "Here is how to make a packet."

An agent can actually create the packet file, render the PDF, generate the video, verify the files, and send them.

6. The kitchen analogy

- **LLM:** the chef's brain and recipe knowledge.
- **Harness:** the kitchen, knives, stove, pantry, safety rules, timer, dishwasher.
- **Tools:** knife, oven, measuring cup, mixer.
- **Memory:** recipe notebook and customer preferences.
- **Agent:** chef actively preparing dinner.
- **Human review:** tasting the food before serving.

A brilliant chef in an empty room cannot cook. A kitchen with no chef cannot cook. You need both.

Part 3 — How I Work

7. What I am in this chat

I am an AI assistant named **Sonya** running through **Hermes Agent** on Telegram.

At a high level:

1. You send a message in Telegram.
2. Hermes receives the message through its gateway.
3. Hermes builds a context package for the model: system rules, user preferences, memory, tools available, and your current request.
4. The LLM generates either:
 - a text response, or
 - a tool call, such as "read this file," "run this command," "make a PDF," "ask a subagent," etc.
5. Hermes executes allowed tool calls and returns the result to the model.
6. The model continues until it has enough verified work to answer you.

7. Hermes sends the final response/file/media back to Telegram.

8. How I was “programmed”

There are several layers.

Layer 1: The base model

The base LLM was trained by its provider on large-scale text/code data. I do not have direct visibility into every detail of that training process. Generally, modern LLM development includes:

- pretraining on huge datasets,
- instruction tuning so the model follows directions,
- safety tuning and preference training,
- evaluation against benchmarks,
- deployment behind an API.

Layer 2: System instructions

This session gives me explicit instructions about how to behave. For you, that includes being direct, grounded, no-bullshit, and not faking progress.

System instructions also tell me how to use tools, when to verify facts, when to ask clarification, and when not to guess.

Layer 3: User profile and memory

Hermes stores durable facts about you, like:

- your timezone,
- your school/work context,
- your preference for easy-read explainers,
- your daily routines,
- rules about reminders and study questions.

This is not the same as the LLM magically remembering. It is stored memory injected into future sessions.

Layer 4: Skills

Hermes has **skills**, which are reusable instruction manuals for specific tasks.

Examples:

- how to use Hermes Agent,
- how to use Claude Code,
- how to use Codex,
- how to make videos,
- how to review code,
- how to work with GitHub.

Before answering your request, I loaded relevant skills for Hermes, Claude Code, Codex, and Manim/video production.

Layer 5: Tools

Tools let me act outside plain text.

In this session, available tools include file reading/writing, terminal commands, PDF/video generation via code, subagents, scheduled jobs, memory, text-to-speech, browser automation, image analysis, and Telegram delivery.

Layer 6: Human oversight

You are still the final authority. For anything risky, destructive, expensive, or external, I should ask first. For educational materials like this packet, you already requested the output, so creating and sending it is within scope.

9. What I can do well

- Explain complex ideas in simpler language.
- Organize information into packets, checklists, plans, and workflows.
- Use files and tools to actually create artifacts.
- Run verification commands.
- Use subagents for parallel work.
- Remember durable preferences through stored memory.
- Help with coding, research, scheduling, writing, and accountability.

10. What I can mess up

- I can hallucinate if I do not verify.
- I can misunderstand vague instructions.
- I can over-trust outdated memory.

- I can make coding mistakes.
- I can claim something is done only if I actually verified it — and if I have not, I need to say so plainly.

Part 4 — Hermes Agent Features

Hermes Agent is the harness/platform currently giving me my “body.”

11. Core Hermes features

1. Provider-agnostic models

Hermes can work with many model providers, including OpenAI, Anthropic, OpenRouter, Gemini, DeepSeek, Nous, local models, and more depending on configuration.

Meaning: Hermes is not locked to one brain. It can swap brains.

2. Messaging gateway

Hermes can run through platforms like:

- Telegram,
- Discord,
- Slack,
- WhatsApp,
- Signal,
- Matrix,
- email,
- SMS,
- API server/webhooks.

That is why you can text me from Telegram instead of using only a terminal.

3. Toolsets

Hermes groups capabilities into toolsets, such as:

- **file:** read, write, search, patch files,
- **terminal:** run shell commands and processes,

- **browser/web:** inspect sites or gather current info,
- **vision:** analyze images,
- **image generation:** generate images,
- **video/TTS:** generate media,
- **memory:** save durable facts,
- **session search:** search old conversations,
- **delegation:** spawn subagents,
- **cron jobs:** scheduled reminders and recurring tasks,
- **messaging:** send messages to connected platforms,
- **GitHub/productivity/smart-home integrations:** depending on what is enabled.

4. Persistent memory

Hermes can store long-term user facts and environment facts. Good memory prevents you from repeating yourself.

Bad memory can become stale, so memory should be compact and durable, not random task progress.

5. Skills

Skills are reusable procedures saved as markdown files. They are like “playbooks” for recurring tasks.

Example: if an agent learns the reliable way to build a PDF/video packet, that workflow can be saved as a skill for later.

6. Subagents/delegation

Hermes can ask isolated subagents to work on pieces of a task. For this packet, subagents researched market analysis, compared harnesses, and drafted a video script.

Analogy: I stayed as the project manager while three research assistants worked in parallel.

7. Cron jobs/scheduled tasks

Hermes can schedule future tasks, like reminders, check-ins, study questions, recurring briefings, or watchdog scripts.

8. Profiles

Hermes can run multiple profiles with separate configs, skills, sessions, and memory. This is useful for separating work, experiments, or different assistants.

9. MCP support

MCP means **Model Context Protocol**. It is a standard-ish way for agents to connect to external tools/services. Think of it like a universal adapter for AI tools.

10. Security controls

Hermes can require approvals for risky commands, redact secrets depending on settings, and restrict which toolsets are enabled.

Important: tool access is powerful. An agent with terminal/file/email access needs guardrails.

Part 5 — OpenClaw

12. What OpenClaw is

OpenClaw is an open-source AI agent/gateway/harness ecosystem. Based on its docs, it is oriented around connecting AI agents to channels/tools and providing agent-harness infrastructure.

A simple way to explain it:

OpenClaw is like a switchboard and control layer for AI agents. It helps connect agents, tools, and communication channels.

13. OpenClaw vs Hermes

They are in the same broad category: **agent harness/platform infrastructure**.

- **Hermes:** strong focus on personal assistant workflows, persistent memory, skills, multi-platform messaging, toolsets, cron, subagents, profiles, and self-improvement.
- **OpenClaw:** broader agent/gateway/harness ecosystem, less mainstream than major coding products, and less widely documented/adopted than Claude Code, Codex, LangGraph, etc.

Your older education note said I was running on OpenClaw. In this current session, the system context says I am running through **Hermes Agent**.

That is a good real-world example: the “assistant personality” can remain similar while the harness/platform changes underneath.

Part 6 — Coding Harnesses and Popular Agent Tools

14. What is a coding harness?

A **coding harness** is an AI-agent environment built for software development.

It usually gives the model:

- repository access,
- code search,
- file editing,
- terminal commands,
- tests,
- Git operations,
- issue/PR workflows,
- sandboxing and permissions,
- project memory/rules.

Analogy:

A normal LLM is a smart person talking about cars. A coding harness puts that person in a garage with tools, parts, diagnostics, and a mechanic's manual.

15. Claude Code

Claude Code is Anthropic's coding agent that works in terminal/IDE workflows.

Good for:

- reading codebases,
- editing files,
- debugging,
- refactoring,
- writing tests,
- running commands,

- reviewing changes.

Analogy: a senior pair programmer in your terminal.

Strengths:

- strong reasoning and code understanding,
- good for large codebases,
- useful for debugging and refactoring.

Risks:

- can make incorrect edits,
- can run commands if permitted,
- still needs tests and human review.

16. OpenAI Codex CLI

Codex CLI is OpenAI's coding agent in the command line.

Good for:

- one-shot coding tasks,
- repo edits,
- debugging,
- test-running,
- local development assistance.

Analogy: ChatGPT with hands on your local code folder.

Strengths:

- terminal-native,
- good for iterative code tasks,
- can inspect and modify files.

Risks:

- needs review,
- shell access can be dangerous,
- quality depends on context and task clarity.

17. GitHub Copilot coding agent

GitHub Copilot is moving beyond autocomplete into agentic work.

Good for:

- GitHub issues,
- PRs,
- repo-based team workflows,
- coding suggestions inside GitHub/VS Code.

Analogy: assigning a GitHub ticket to an AI teammate.

18. Cursor and Windsurf

These are AI-first coding editors.

Good for:

- developers who prefer graphical IDE workflows,
- inline code editing,
- project-aware chat,
- agent mode inside editor.

Analogy: VS Code with an AI pair programmer built into the editor.

19. OpenHands / OpenDevin

OpenHands is an open-source software-development agent platform.

Good for:

- experimenting with autonomous coding,
- self-hosting,
- Devin-like workflows,
- research/development.

Analogy: an open-source robot developer in a sandbox.

20. Devin

Devin is a commercial autonomous software-engineering agent.

Good for:

- teams assigning coding tasks to cloud agents,
- parallel engineering tasks,
- more autonomous ticket completion.

Analogy: a remote AI contractor working in its own environment.

21. Frameworks for building agents

These are less “ready-made assistant” and more “developer toolkit.”

LangGraph

Best for reliable, stateful, production agent workflows.

Analogy: a flowchart/circuit board for agent behavior.

AutoGen

Best for multi-agent conversations and orchestration.

Analogy: a room of AI specialists talking through a problem.

CrewAI

Best for role-based agents like researcher, writer, reviewer.

Analogy: a small AI team with job titles.

LlamaIndex agents

Best for agents over documents/private knowledge.

Analogy: a librarian-agent that searches your knowledge base.

Haystack

Best for modular RAG/search pipelines and enterprise knowledge systems.

Analogy: Lego blocks for search-and-answer AI.

Semantic Kernel

Best for Microsoft/.NET/Azure enterprise apps.

Analogy: a toolkit for adding AI functions to normal software.

Smolagents

Best for lightweight/simple open-source agent experiments.

Analogy: a small toolbox instead of a giant construction site.

Part 7 — Market Analysis: AI Agents

22. What is happening in the market?

The market is moving through stages:

Stage 1: Chatbots

People asked AI questions and got answers.

Examples: ChatGPT, Claude, Gemini.

Stage 2: Copilots

AI sits beside a human and helps them work.

Examples: GitHub Copilot, Microsoft Copilot, writing assistants.

Stage 3: Agents

AI starts taking action through tools.

Examples: Claude Code, Codex, Devin, Copilot coding agent, Hermes scheduled workflows.

Stage 4: Agent ecosystems

Companies build platforms where many agents can work across departments, software systems, and communication channels.

Examples: Salesforce Agentforce, Microsoft Copilot Studio, AWS Bedrock Agents, Google Gemini Enterprise, ServiceNow AI Agents.

23. Major categories

Foundation model companies

They build the brains:

- OpenAI,
- Anthropic,
- Google DeepMind,
- Meta,
- Mistral,
- DeepSeek,
- xAI,
- Cohere,
- Alibaba/Qwen.

Agent platforms

They build the harnesses and developer infrastructure:

- OpenAI Agents SDK,
- LangGraph,
- AutoGen,
- CrewAI,
- LlamaIndex,
- Semantic Kernel,
- Haystack,
- Hermes,
- OpenClaw.

Coding agents

They focus on software engineering:

- Claude Code,
- Codex,
- GitHub Copilot coding agent,
- Cursor,

- Windsurf,
- Devin,
- Google Jules,
- OpenHands.

Enterprise agents

They automate business workflows:

- Salesforce Agentforce,
- Microsoft Copilot Studio,
- AWS Bedrock Agents,
- Google Gemini Enterprise/Agentspace,
- ServiceNow AI Agents,
- UiPath.

24. Why coding agents are moving fastest

Coding is a strong early use case because:

- code is text,
- code has tests,
- Git tracks changes,
- errors can be verified,
- tasks can be scoped as tickets,
- companies already pay a lot for software labor.

This makes coding agents easier to measure than vague tasks like “make my company more efficient.”

25. Why enterprise agents are huge

Companies want agents to handle:

- customer service,
- sales follow-ups,
- IT tickets,
- HR questions,
- finance operations,

- internal knowledge search,
- workflow automation.

But enterprise adoption is slower because of:

- privacy,
- legal risk,
- security,
- reliability,
- integration complexity,
- governance.

26. Market opportunities

Big opportunities:

1. **Personal AI assistants** that actually know your life and preferences.
2. **Coding agents** that reduce repetitive engineering work.
3. **Healthcare/legal/education support** with strict oversight.
4. **Small business automation** for emails, scheduling, documents, invoices.
5. **Enterprise workflow agents** connected to existing software.
6. **Agent safety/monitoring tools** because companies need guardrails.
7. **Evaluation tools** to measure whether agents are actually doing good work.

27. Market risks and hype

The risk is that companies call everything an “agent” even when it is just a chatbot with branding.

Common failure points:

- unclear ROI,
- too much autonomy too soon,
- weak security,
- hallucinations,
- prompt injection,
- bad integrations,
- no monitoring,
- no human approval path,

- expensive API usage,
- agents that look impressive in demos but fail in messy real workflows.

A grounded view:

Agents are real and useful, but the winners will be systems that combine good models with strong harnesses, verification, permissions, monitoring, and human review.

Part 8 — How to Explain This to Someone New

28. The 30-second version

“An LLM is the AI brain that understands and generates language. A harness is the software body around it that gives it tools, memory, rules, and access to things like files or the internet. When you combine the brain with a harness and give it a goal, you get an AI agent. So Sonya is not just a chatbot — she is an LLM running inside Hermes, which lets her use tools, remember preferences, create files, schedule reminders, and send things through Telegram.”

29. The 2-minute version

“Think of AI like hiring a smart assistant. The LLM is their brain. But a brain alone cannot do much unless it has a workplace. The harness is the workplace: computer, files, internet, calendar, rules, safety approvals, and memory. An agent is the assistant using that workplace to complete a goal.

So ChatGPT by itself is often just conversation. But when an LLM is placed inside something like Hermes, Claude Code, Codex, or OpenClaw, it can use tools. Hermes lets Sonya talk through Telegram, read/write files, use subagents, remember preferences, schedule reminders, and create artifacts like PDFs or videos. Coding harnesses like Claude Code and Codex do the same idea but focused on software: they read code, edit files, run tests, and help with GitHub.

The big market shift is that AI is moving from answering questions to doing tasks. The challenge is making sure these agents are accurate, safe, monitored, and useful instead of just flashy.”

30. The easiest analogy

“LLM is the engine. Harness is the car. Agent is the car driving toward a destination. Human oversight is the driver/instructor making sure it does not crash.”

Better version:

- Engine = model,
- Car body = harness,
- Steering wheel/brakes = permissions,
- GPS = goal/planning,
- Dashboard = logs/status,
- Mechanic = human review,
- Destination = completed task.

31. How to explain me specifically

“Sonya is an AI assistant running through Hermes Agent. The model generates language and decides what to do next. Hermes gives her tools: Telegram, files, terminal, memory, skills, scheduled jobs, and subagents. When I ask Sonya for something, Hermes gives the model my request plus relevant instructions and tools. If the job requires action, the model calls tools through Hermes, sees the results, and continues until it can send me the final answer or file.”

Part 9 — Practical Mental Models

32. Treat AI agents like fast junior employees

They are useful, but they need:

- clear instructions,
- access only to what they need,
- examples,
- checklists,
- review,
- boundaries,
- verification.

Do not treat them like omniscient gods. Treat them like very fast interns who can read and type at super speed but still need supervision.

33. Good prompts for agents

Bad prompt:

“Make this better.”

Good prompt:

“Create a beginner-friendly 10-page packet explaining LLMs vs harnesses. Include analogies, a glossary, market overview, examples of Claude Code/Codex/Hermes/OpenClaw, and export as PDF. Verify the file exists before sending.”

Strong agent prompts include:

- goal,
- audience,
- format,
- constraints,
- examples,
- verification requirement,
- what not to do.

34. When to trust vs verify

Trust more when:

- the task is low risk,
- the agent shows sources,
- the output can be checked,
- tests pass,
- facts are current and cited.

Verify aggressively when:

- money, health, legal, or safety is involved,
- the agent used tools with side effects,
- code changes are involved,
- credentials or private data are involved,
- the answer includes current market/news claims.

Part 10 — Glossary

Agent: AI system that uses a model plus tools to pursue a goal.

API: A way for software systems to talk to each other.

API key: A password for software services.

Claude Code: Anthropic's coding agent/harness.

Codex CLI: OpenAI's coding agent/harness.

Context window: How much text the model can consider at once.

Cron job: Scheduled task that runs later or repeatedly.

Embedding: Numeric representation of meaning used for search/retrieval.

Fine-tuning: Additional training to specialize a model.

Harness: The wrapper/control system giving an LLM tools, memory, rules, and execution.

Hermes Agent: The agent platform/harness currently running this assistant.

Hallucination: Confident but incorrect AI output.

LLM: Large Language Model; the language "brain."

MCP: Model Context Protocol; a way to connect models/agents to tools.

OpenClaw: Agent/gateway/harness ecosystem.

Prompt injection: Malicious text that tries to trick an AI into ignoring rules.

RAG: Retrieval-Augmented Generation; AI searches documents then answers from them.

Subagent: A separate agent assigned a smaller task.

Tool call: When an AI asks the harness to run a tool, like reading a file or searching web.

Part 11 — Sources and Further Reading

- IBM overview of AI agents: <https://www.ibm.com/think/topics/ai-agents>
- OpenAI Agents SDK/Responses API announcement: <https://openai.com/index/new-tools-for-building-agents/>
- OpenAI Codex: <https://openai.com/index/introducing-codex/>
- OpenAI Codex GitHub: <https://github.com/openai/codex>
- Anthropic Claude Code: <https://www.anthropic.com/product/claude-code>
- Claude Code docs: <https://docs.anthropic.com/en/docs/claude-code/overview>
- Hermes Agent docs: <https://hermes-agent.nousresearch.com/docs/>
- Hermes Agent GitHub: <https://github.com/NousResearch/hermes-agent>
- OpenClaw docs: <https://docs.openclaw.ai/>
- OpenClaw agent harness docs: <https://docs.openclaw.ai/plugins/sdk-agent-harness>
- GitHub Copilot coding agent docs: <https://docs.github.com/en/copilot/concepts/coding-agent/coding-agent>
- Google Jules: <https://jules.google/>

- Cursor docs: <https://docs.cursor.com/>
- Windsurf docs: <https://docs.windsurf.com/windsurf/getting-started>
- OpenHands: <https://github.com/All-Hands-AI/OpenHands>
- OpenHands docs: <https://docs.openhands.dev/>
- LangGraph docs: <https://docs.langchain.com/oss/python/langgraph/overview>
- AutoGen docs: <https://microsoft.github.io/autogen/stable/>
- CrewAI docs: <https://docs.crewai.com/>
- LlamaIndex agents: <https://developers.llamaindex.ai/python/framework/understanding/agent/>
- Haystack: <https://haystack.deepset.ai/>
- Semantic Kernel: <https://learn.microsoft.com/en-us/semantic-kernel/overview/>
- Hugging Face Smolagents: <https://huggingface.co/docs/smolagents/en/index>
- AWS Bedrock Agents: <https://aws.amazon.com/bedrock/agents/>
- Microsoft Copilot Studio: <https://learn.microsoft.com/en-us/microsoft-copilot-studio/>
- Salesforce Agentforce: <https://www.salesforce.com/agentforce/>
- ServiceNow AI Agents: <https://www.servicenow.com/products/ai-agents.html>
- NIST AI Risk Management Framework: <https://www.nist.gov/itl/ai-risk-management-framework>
- OWASP Top 10 for LLM Applications:
<https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- Grand View Research enterprise agentic AI market report:
<https://www.grandviewresearch.com/industry-analysis/enterprise-agentic-ai-market-report>
- ResearchAndMarkets agentic AI market forecast:
<https://www.researchandmarkets.com/reports/6160154/agentic-ai-market-global-forecast>

Final Takeaway

If you remember only one thing, remember this:

The LLM is the brain. The harness is the body, tools, memory, and rules. The agent is the whole system trying to complete a goal.

That is how I work. I am not just “a chatbot in the air.” I am a model running inside Hermes, with tools, memory, skills, subagents, scheduled jobs, and Telegram delivery. That harness is what lets me create this packet and video instead of merely explaining how one could be made.